Real Time UAV Tracking using The Unscented Kalman Filter on a Dual-Axis Optical System

Ivar Ødegårdstuen Eftedal
Faculty of Science and Technology
Norwegian University of Life Sciences
ivef@nmbu.no

***Abstract*—Passive optical tracking of Unmanned Aerial Vehicles (UAVs) is a core requirement for counter-UAS systems, where a ground-based camera must continuously estimate the position and velocity of an intruder drone from noisy bearing-only measurements. This paper presents a real-time trajectory estimation system using a Square-Root Unscented Kalman Filter (SR-UKF) that bypasses the linearization errors inherent to the Extended Kalman Filter, enabling robust tracking of agile drones through aggressive manoeuvres.**

Developed using the C++23 standard, the implementation leverages constexpr matrix generation and compile-time policy dispatch to achieve low-latency execution without external dependency overhead. The SR-UKF uses the scaled unscented transform to propagate state distributions through non-linear bearing-only measurement functions, capturing the true pan and tilt posterior to second-order accuracy.

Evaluation on simulated flight trajectories demonstrates consistent convergence with Normalised Estimation Error Squared (NEES) metrics within chi-squared confidence bounds across Monte Carlo trials. The implementation achieves deterministic sub-microsecond execution times suitable for real-time gimbal control on embedded edge hardware.

I. Introduction

The proliferation of consumer drones has created an urgent need for counter-UAS (C-UAS) systems capable of detecting, tracking, and mitigating unauthorised aerial intrusions over sensitive airspace. Airports, critical infrastructure, and public events increasingly face threats from small, agile UAVs that are difficult to detect with traditional radar due to their low radar cross-section and slow speed. Passive optical tracking—using a camera mounted on a dual-axis pan/tilt gimbal—offers a low-cost, non-emitting alternative: the system measures bearing angles (pan and tilt) to the target and must estimate the underlying 3-D position and velocity from this sequence of noisy, non-linear measurements. Any lag or divergence in the state estimate causes the camera to lose the target, breaking the tracking lock and rendering the system ineffective.

The standard estimation approach for this class of problem is the Extended Kalman Filter (EKF), which linearises the non-linear bearing-only measurement function via first-order Taylor expansion (Jacobians). When an intruder drone executes a sharp turn, rapid climb, or fast acceleration, this linearised approximation breaks down: the Gaussian distribution propagated through the approximate function no longer represents the true posterior, producing biased pan/tilt predictions that cause the gimbal to point off-target. Compounding the mathematical limitation, C-UAS software has historically relied on legacy C++ libraries with manual memory management or on Python-based prototyping frameworks. For a real-time gimbal controller operating at camera frame rate, non-deterministic latency or memory corruption is unacceptable—a missed frame can mean losing a track that takes seconds to reacquire.

This project addresses both the algorithmic and the software engineering challenge simultaneously. On the estimation side, we replace the EKF's Jacobian-based linearisation with the Unscented Kalman Filter (UKF). The UKF uses a deterministic sampling approach—the scaled unscented transform—to propagate the full state distribution through the true non-linear bearing-only measurement equations without any local approximation, capturing the posterior pan/tilt mean and covariance to second-order accuracy. The system supports both constant-velocity (6-D) and constant-acceleration (9-D) kinematic models in the inertial ground frame using a Square-Root UKF (SR-UKF) formulation, with pan and tilt as the observed measurements.

On the engineering side, the entire filter is implemented in C++23, deliberately chosen for its zero-cost abstractions, compile-time computation, and strict type safety. A policy-based template architecture dispatches process models and measurement models at compile time via template specialisation, eliminating virtual dispatch and enabling the compiler to inline and optimise the entire filter pipeline. The library is header-only, depends solely on Eigen 3.4+ for linear algebra, and is validated on both bearing-only (pan/tilt) and linear pixel-space measurement models at 9-D, 6-D, and 4-D state dimensions. A real-time video tracking application integrates the UKF with a YOLOv8 object detector, demonstrating continuous track maintenance through occlusions when the detector temporarily loses the target.

The primary contributions of this work are as follows:

- **High-Fidelity State Estimation for C-UAS**: A complete Unscented Kalman Filter architecture tuned for bearing-only tracking of non-linear UAV trajectories, supporting both constant-velocity and constant-acceleration process models with compile-time dispatch.
- **Modernized C++23 Software Design**: An implementation completely free of legacy raw pointers, leveraging constexpr for compile-time filter tuning parameters and model matrix generation, policy-based compile-

time polymorphism, and Eigen 3.4+ for zero-overhead linear algebra.
- **Empirical Validation**: A test suite including Monte Carlo NEES consistency checks (50-run ensembles), convergence verification against figure-8 trajectories, and Google Benchmark measurements demonstrating deterministic sub-microsecond execution on commodity hardware.
- **Bridging Theory and Practice**: Combining second-order non-linear estimation accuracy with a compile-time execution architecture, this work demonstrates that rigorous estimation-theoretic principles—including observability analysis, consistent covariance propagation, and statistically valid gating—can be realised in a real-time C-UAS system without sacrificing computational performance.

The remainder of this report is structured as follows. Section 2 derives the UKF algorithm and presents the process and measurement models. Section 3 documents the C++23 software architecture. Section 4 presents empirical results, which are discussed in Section 5. Section 6 concludes and outlines future work.

II. Mathematical Framework

This section derives the Unscented Kalman Filter, defines the kinematic process models, and presents the bearing-only measurement model used for passive optical tracking. All models are implemented at compile time via C++ templates; the corresponding software architecture is detailed in Section 3.

A. Notation

State	
$\boldsymbol{x}_k \in \mathbb{R}^n$	State vector at discrete time k.
p_x, p_y, p_z	Position components (m).
v_x, v_y, v_z	Velocity components (m/s).
a_x, a_y, a_z	Acceleration components (m/s^2).
n	State dimension (6 for CV, 9 for CA).
Process Model	
$\boldsymbol{F}(\Delta t) \in \mathbb{R}^{n\times n}$	State-transition matrix.
$\boldsymbol{Q}(\Delta t, q_c) \in \mathbb{R}^{n\times n}$	Process-noise covariance.
Δt	Sampling interval (s).
q_c	Power spectral density of the continuous-time jerk process.
$\boldsymbol{w}(t)$	Continuous-time white-noise jerk process.
$\hat{\boldsymbol{x}}_k^-$	Predicted (a priori) state estimate at k.
$\boldsymbol{P}_k^-$	Predicted (a priori) covariance at k.
$\hat{\boldsymbol{x}}_k$	Posterior state estimate at k.
$\boldsymbol{P}_k$	Posterior state covariance at k.
Measurement Model	
$\boldsymbol{z}_k \in \mathbb{R}^m$	Measurement vector at discrete time k.
m	Measurement dimension (2 for pan/tilt, 2 for pixel).
θ	Pan angle (horizontal bearing, rad).
φ	Tilt angle (elevation, rad).
$h(\boldsymbol{x})$	Non-linear bearing measurement function.
$\boldsymbol{R} \in \mathbb{R}^{m\times m}$	Measurement noise covariance.
$\boldsymbol{H} \in \mathbb{R}^{m\times n}$	Measurement Jacobian (linear model).
u, v	Pixel coordinates.
ν_θ	Pan innovation residual.
Unscented Transform	
$\boldsymbol{X}_i$	Sigma points, $i = 0, ..., 2n$.
λ	Composite scaling parameter: $\lambda = \alpha^2(n+\kappa) - n$.
α	Primary sigma-point spread parameter.
β	Prior distribution parameter ($\beta = 2$ optimal for Gaussian).
κ	Secondary scaling parameter.
w_m^i	Weight for mean recovery at sigma point i.
w_c^i	Weight for covariance recovery at sigma point i.
$\boldsymbol{Z}_i$	Sigma point i propagated through $h(\cdots)$.
$\hat{\boldsymbol{z}}$	Predicted measurement mean.
$\boldsymbol{S} \in \mathbb{R}^{m\times m}$	Innovation covariance.
$\boldsymbol{C} \in \mathbb{R}^{n\times m}$	State-measurement cross-covariance.
$\boldsymbol{K} \in \mathbb{R}^{n\times m}$	Kalman gain.
Square-Root UKF	
$\boldsymbol{S}_k$	Cholesky factor: $\boldsymbol{P}_k = \boldsymbol{S}_k \boldsymbol{S}_k^T$.
$\boldsymbol{S}_{\boldsymbol{z}}$	Innovation Cholesky factor.
Evaluation	
ε_k	Normalised Estimation Error Squared (NEES) at step k.
χ^2_d	Chi-squared distribution with d degrees of freedom.

B. State Representation

The target state is estimated in the inertial ground frame $\mathcal{F}^G$ (see Section 2.3), with the state vector comprising position, velocity, and optionally acceleration at discrete time k.

For the constant-acceleration (CA) model the state dimension is $n = 9$:

$$\boldsymbol{x}_k = \begin{pmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \\ a_x \\ a_y \\ a_z \end{pmatrix} \tag{1}$$

where p denotes 3-D position (in meters), v the linear velocity (m/s), and a the linear acceleration (m/s^2). For the constant-velocity (CV) model the dimension is $n = 6$ and acceleration is omitted.

a) *Coordinate Frames and Gimbal Kinematics:*

A rigorous treatment of bearing-only tracking on a gimbal requires the explicit definition of three coordinate frames. The tracking is performed in a stationary **Inertial Ground Frame** $\mathcal{F}^G$, where position and velocity are physically meaningful. The gimbal base defines the **Gimbal Base Frame** $\mathcal{F}^B$, which is rigidly attached to the ground with a known fixed rotation $\boldsymbol{R}_G^B$ and translation $\boldsymbol{t}_G^B$. The camera defines the **Camera Frame** $\mathcal{F}^C$, whose orientation relative to the gimbal base is determined by the pan angle θ and tilt angle φ at each time instant.

All state estimation in this work is performed in $\mathcal{F}^G$, an inertial Earth-fixed frame. The pan/tilt measurements $\boldsymbol{z}_k = \begin{pmatrix} \theta \\ \varphi \end{pmatrix}$ are related to the target position in the ground frame via the composition of two transformations. A point $\boldsymbol{p}^G$ in the ground frame is first transformed into the camera frame by the instantaneous gimbal pose:

$$\boldsymbol{p}^C = \boldsymbol{R}_B^C(\theta_k, \varphi_k)\boldsymbol{R}_G^B\big(\boldsymbol{p}^G - \boldsymbol{t}_G^B\big) \tag{2}$$

where $\boldsymbol{R}_B^C(\theta, \varphi)$ is the rotation induced by the pan and tilt servos. The bearing measurement is then the direction of $\boldsymbol{p}^C$ expressed in the camera frame:

$$\theta_k = \operatorname{atan2}\big(p_x^C, p_z^C\big), \quad \varphi_k = \operatorname{atan2}\left(p_y^C, \sqrt{\big(p_x^C\big)^2 + \big(p_z^C\big)^2}\right) \tag{3}$$

Because $\boldsymbol{R}_G^B$, $\boldsymbol{t}_G^B$, and the instantaneous θ_k, φ_k are all known—from calibration and encoder readings—the mapping is fully deterministic. The filter therefore estimates the inertial state $\boldsymbol{x}_k$ directly, and the non-linear measurement function $h(\cdots)$ implicitly accounts for the gimbal kinematics through these known transformations. Since the gimbal rotation is treated as a known deterministic input rather than an additional stochastic state, no fictitious forces or Coriolis terms appear in the state-transition equations.

C. Process Model

The continuous-time kinematic model for the CA case is a Wiener-process acceleration (jerk) model:

$$\dot{\boldsymbol{x}}(t) = \begin{pmatrix} v_{x(t)} \\ v_{y(t)} \\ v_{z(t)} \\ a_{x(t)} \\ a_{y(t)} \\ a_{z(t)} \\ w_{x(t)} \\ w_{y(t)} \\ w_{z(t)} \end{pmatrix}, \quad \boldsymbol{w}(t) \sim \mathcal{N}(0, q_c \boldsymbol{I}_3) \tag{4}$$

where $\boldsymbol{w}(t)$ is a continuous-time white-noise jerk process with power spectral density q_c. Following the discrete-time white-noise jerk (piecewise white-noise) model [1], the continuous-time noise is assumed constant over each sampling interval. Exact discretisation with time step Δt yields the discrete-time state-transition matrix:

$$\boldsymbol{F}(\Delta t) = \begin{pmatrix} \boldsymbol{I}_3 & \Delta t \boldsymbol{I}_3 & \frac{1}{2}\Delta t^2 \boldsymbol{I}_3 \\ \boldsymbol{0}_3 & \boldsymbol{I}_3 & \Delta t \boldsymbol{I}_3 \\ \boldsymbol{0}_3 & \boldsymbol{0}_3 & \boldsymbol{I}_3 \end{pmatrix} \tag{5}$$

and the process-noise covariance:

$$\boldsymbol{Q}(\Delta t, q_c) = q_c \begin{pmatrix} \frac{\Delta t^5}{20}\boldsymbol{I}_3 & \frac{\Delta t^4}{8}\boldsymbol{I}_3 & \frac{\Delta t^3}{6}\boldsymbol{I}_3 \\ \frac{\Delta t^4}{8}\boldsymbol{I}_3 & \frac{\Delta t^3}{3}\boldsymbol{I}_3 & \frac{\Delta t^2}{2}\boldsymbol{I}_3 \\ \frac{\Delta t^3}{6}\boldsymbol{I}_3 & \frac{\Delta t^2}{2}\boldsymbol{I}_3 & \Delta t \boldsymbol{I}_3 \end{pmatrix} \tag{6}$$

For the CV model ($n = 6$) the corresponding matrices are:

$$\begin{aligned} \boldsymbol{F}_{\mathrm{CV}} &= \begin{pmatrix} \boldsymbol{I}_3 & \Delta t \boldsymbol{I}_3 \\ \boldsymbol{0}_3 & \boldsymbol{I}_3 \end{pmatrix}, \\ \boldsymbol{Q}_{\mathrm{CV}} &= q_c \begin{pmatrix} \frac{\Delta t^3}{3}\boldsymbol{I}_3 & \frac{\Delta t^2}{2}\boldsymbol{I}_3 \\ \frac{\Delta t^2}{2}\boldsymbol{I}_3 & \Delta t \boldsymbol{I}_3 \end{pmatrix} \end{aligned} \tag{7}$$

The discrete-time prediction step is:

$$\hat{\boldsymbol{x}}_k^- = \boldsymbol{F}\hat{\boldsymbol{x}}_{k-1} \tag{8}$$

$$\boldsymbol{P}_k^- = \boldsymbol{F}\boldsymbol{P}_{k-1}\boldsymbol{F}^T + \boldsymbol{Q} \tag{9}$$

D. Measurement Model

The camera provides bearing-only measurements $\boldsymbol{z}_k = \begin{pmatrix} \theta \\ \varphi \end{pmatrix}$ where θ is the pan angle (horizontal bearing) and φ is the tilt angle (elevation). The non-linear measurement function maps the camera-centric position to bearings:

$$h(\boldsymbol{x}_k) = \begin{pmatrix} \operatorname{atan2}(p_x, p_z) \\ \operatorname{atan2}\big(p_y, \sqrt{p_x^2 + p_z^2}\big) \end{pmatrix} \tag{10}$$

Pan angles are defined on $(-\pi, \pi]$ and tilt is bounded to $\left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$.

In addition to the bearing-only model, the system also implements a linear pixel-space measurement model for 2-D video tracking where $h(\boldsymbol{x}_k) = \begin{pmatrix} u \\ v \end{pmatrix}$ extracts pixel coordinates directly from the state vector. For this case the measurement Jacobian $\boldsymbol{H} = (\boldsymbol{I}_2 \;\; \boldsymbol{0}_2 \;\; \cdots)$ is constant, and the standard closed-form Kalman update is used instead of sigma-point propagation.

a) *Observability of Bearing-Only Tracking:*

A critical aspect of bearing-only tracking (BOT) that distinguishes it from full-position measurement is the observability of the target range. For a stationary observer, the bearing-only system is not fully observable: range cannot be determined from bearing measurements alone when the target moves with constant velocity. This limitation is not merely a practical nuisance but a structural property of the measurement model.

To analyse this formally, consider the continuous-time kinematic model for the 6-D CV case. The state is $\boldsymbol{x} = \begin{pmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{pmatrix}^T$ and the measurement is $\boldsymbol{z} = \begin{pmatrix} \theta & \varphi \end{pmatrix}^T$. Defining the drift and measurement vector fields for the non-linear system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$, $\boldsymbol{z} = \boldsymbol{h}(\boldsymbol{x})$:

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{pmatrix} v_x \\ v_y \\ v_z \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{h}(\boldsymbol{x}) = \begin{pmatrix} \operatorname{atan2}(p_x, p_z) \\ \operatorname{atan2}\!\left(p_y, \sqrt{p_x^2 + p_z^2}\right) \end{pmatrix} \tag{11}$$

The observability codistribution is spanned by the gradients of successive Lie derivatives of the measurement function along the drift field:

$$\mathcal{L}_{\boldsymbol{f}}^0 \boldsymbol{h} = \boldsymbol{h}, \quad \mathcal{L}_{\boldsymbol{f}}^1 \boldsymbol{h} = \nabla \boldsymbol{h} \cdot \boldsymbol{f} \tag{12}$$

Computing these gradients reveals that the unobservable subspace corresponds to trajectories where the line-of-sight (LOS) vector $\boldsymbol{p}$ is collinear with the velocity vector $\boldsymbol{v}$: $\boldsymbol{v} \propto \boldsymbol{p}$. In this regime, bearing rates are zero and no range information enters the measurements. Range becomes observable only when the relative geometry changes—i.e., when the bearing rate is non-zero, which requires either observer motion (egomotion) or target manoeuvres that change the LOS direction.

For a ground-based stationary gimbal, the observer has no egomotion. The filter therefore relies entirely on the process model's velocity dynamics to infer range indirectly: the coupling between position and velocity in the state-transition matrix $\boldsymbol{F}$ propagates velocity information into the position estimate during the predict step. This provides **weak observability** when the target bearing changes (i.e., when the target has non-zero lateral velocity), but the range estimate remains highly sensitive to the process noise tuning q_c and the initial covariance $\boldsymbol{P}_0$. In practice, the range error covariance may remain large even when bearing errors are small, a behaviour confirmed by the simulation results in Section 4.

E. Unscented Transform

The core of the UKF is the unscented transform, which propagates a set of deterministically chosen sigma points through the non-linear measurement function $h(\cdots)$ and recovers the posterior statistics to second-order accuracy. Given an n-dimensional state with mean $\hat{\boldsymbol{x}}$ and covariance $\boldsymbol{P}$, the $2n+1$ sigma points are:

$$\boldsymbol{X}_0 = \hat{\boldsymbol{x}} \tag{13}$$

$$\boldsymbol{X}_i = \hat{\boldsymbol{x}} + \left(\sqrt{(n+\lambda)\boldsymbol{P}}\right)_i, \quad i = 1, ..., n \tag{14}$$

$$\boldsymbol{X}_{i+n} = \hat{\boldsymbol{x}} - \left(\sqrt{(n+\lambda)\boldsymbol{P}}\right)_i, \quad i = 1, ..., n \tag{15}$$

where the matrix square root is obtained from the propagated Cholesky factor in the SR-UKF implementation (Section 3.2). The scaling parameters are:

$$\lambda = \alpha^2(n + \kappa) - n \tag{16}$$

with α controlling sigma-point spread (typically 10^{-3}), β encoding prior knowledge ($\beta = 2$ optimal for Gaussian distributions), and κ a secondary scaling parameter.

The weights for mean and covariance recovery are:

$$\begin{aligned} w_m^0 &= \frac{\lambda}{n+\lambda}, \\ w_c^0 &= w_m^0 + (1 - \alpha^2 + \beta) \end{aligned} \tag{17}$$

$$w_m^i = w_c^i = \frac{1}{2(n+\lambda)}, \quad i = 1, ..., 2n \tag{18}$$

Each sigma point is propagated through the measurement function:

$$\boldsymbol{Z}_i = h(\boldsymbol{X}_i), \quad i = 0, ..., 2n \tag{19}$$

The predicted measurement mean, innovation covariance, and cross-covariance are then:

$$\hat{\boldsymbol{z}} = \sum_{i=0}^{2n} w_m^i \boldsymbol{Z}_i \tag{20}$$

$$\boldsymbol{S} = \sum_{i=0}^{2n} w_c^i (\boldsymbol{Z}_i - \hat{\boldsymbol{z}})(\boldsymbol{Z}_i - \hat{\boldsymbol{z}})^T + \boldsymbol{R} \tag{21}$$

$$\boldsymbol{C} = \sum_{i=0}^{2n} w_c^i (\boldsymbol{X}_i - \hat{\boldsymbol{x}})(\boldsymbol{Z}_i - \hat{\boldsymbol{z}})^T \tag{22}$$

where $\boldsymbol{R}$ is the measurement noise covariance. The Kalman gain and posterior update are:

$$\boldsymbol{K} = \boldsymbol{C}\boldsymbol{S}^{-1} \tag{23}$$

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + \boldsymbol{K}(\boldsymbol{z}_k - \hat{\boldsymbol{z}}) \tag{24}$$

$$\boldsymbol{P}_k = \boldsymbol{P}_k^- - \boldsymbol{K}\boldsymbol{S}\boldsymbol{K}^T \tag{25}$$

In the SR-UKF implementation (Section 3.2), the Kalman gain is obtained via efficient triangular back-substitution using the precomputed Cholesky factor $\boldsymbol{S}_{\boldsymbol{z}}$, which is numerically more stable and computationally cheaper than explicit matrix inversion. An innovation-wrapping step places the pan innovation ν_θ in $(-\pi, \pi]$ via $\nu_\theta = \text{remainder}(\nu_\theta, 2\pi)$ to prevent angular wraparound artefacts.

For linear measurement models (isMeasurementLinear == true), the sigma-point propagation is bypassed entirely. The standard Kalman filter update is used:

$$\boldsymbol{K} = \boldsymbol{P}_k^- \boldsymbol{H}^T \left(\boldsymbol{H}\boldsymbol{P}_k^- \boldsymbol{H}^T + \boldsymbol{R}\right)^{-1} \tag{26}$$

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + \boldsymbol{K}(\boldsymbol{z}_k - \boldsymbol{H}\hat{\boldsymbol{x}}_k^-) \tag{27}$$

$$\boldsymbol{P}_k = (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}_k^- \tag{28}$$

III. System Architecture

A. Software Design

The entire filter is implemented as a C++23 header-only library with optional pre-compiled template instantiations. The architecture follows a policy-based design pattern where compile-time template parameters select both the process model and the measurement model, eliminating all virtual dispatch overhead and enabling aggressive inlining by the compiler.

The primary class template is:

```
template <typename T, int N, int M, typename Policy =
PanTiltPolicy>
class UnscentedKalmanFilter;
```

where T is the scalar type (float or double), N and M are the state and measurement dimensions, and Policy provides:

- ModelTag: an empty tag type used to dispatch to the correct process model specialisation at compile time
- h(x): the measurement function (non-linear for pan/tilt, linear for pixel space)
- H(x): the measurement Jacobian (used only when isMeasurementLinear is true)
- wrapInnovation(nu): wraps angular residuals
- isMeasurementLinear: a static constexpr bool that selects between the full UKF sigma-point update and the closed-form Kalman update

Process models specialise a primary template:

```
template <typename T, int N, typename Tag>
struct ProcessModel {
  static constexpr auto F(T dt) noexcept { ... }
  static constexpr auto Q(T dt, T q_c) noexcept { ... }
};
```

Specialisations exist for $N = 9$ (CA), $N = 6$ (CV), $N = 4$ (pixel CV), and $N = 6$ with PixelCATag (pixel CA). The Tag parameter allows multiple models with identical N to coexist without ambiguity.

a) *C++23 Features:*

The implementation deliberately uses the C++23 standard for the following engineering benefits:

- **Compile-time computation**: All process model matrices $\boldsymbol{F}(\Delta t)$ and $\boldsymbol{Q}(\Delta t, q_c)$ are constexpr functions. When the time step is known at compile time, the matrices are computed entirely by the compiler with zero run-time overhead. In practice, frame-to-frame timing jitter in camera systems means Δt is measured at runtime via high-resolution clocks; the constexpr path serves as an optimisation for constant-frame-rate operation, while the runtime path gracefully handles variable timing by evaluating the same functions at call time.
- **Policy-based compile-time polymorphism**: The if constexpr branching on Policy::isMeasurementLinear selects either the UKF or KF update path at compile time, eliminating branch misprediction and dead code.
- **Type-safe scalar abstraction**: The template parameter T permits single-precision (float) deployment on resource-constrained hardware with no code duplication, while double provides full precision for offline analysis.
- **No raw pointers**: All matrix storage uses Eigen 3.4+'s RAII types, eliminating manual memory management and the associated safety bugs.
- **Debug-mode assertions**: assert(x_.allFinite()) guards in every predict and update step, catching numerical divergence during development without affecting release performance.

B. Data Flow

The filter pipeline operates in three stages:

1) **Configuration**: The user sets the process noise spectral density q_c, the measurement noise covariance $\boldsymbol{R}$, the initial state $\hat{\boldsymbol{x}}_0$, and the initial covariance $\boldsymbol{P}_0$. The UKF scaling parameters α, β, κ are set at construction.
2) **Time update (predict)**: At each time step Δt, the filter computes $\hat{\boldsymbol{x}}_k^- = \boldsymbol{F}\hat{\boldsymbol{x}}_{k-1}$ and $\boldsymbol{P}_k^- = \boldsymbol{F}\boldsymbol{P}_{k-1}\boldsymbol{F}^T + \boldsymbol{Q}$. The matrices $\boldsymbol{F}$ and $\boldsymbol{Q}$ are selected at compile time based on N and ModelTag; their elements are computed from Δt and q_c by the constexpr functions.
3) **Measurement update (correct)**: When a new bearing measurement $\boldsymbol{z}_k = \begin{pmatrix}\theta \\ \varphi\end{pmatrix}$ arrives, the filter propagates sigma points through $h(\cdots)$, computes the innovation $\boldsymbol{z}_k - \hat{\boldsymbol{z}}$, wraps the pan residual, and updates the state and covariance via the Kalman gain. For pixel-space tracking, the linear update path is taken.

When no measurement is available (e.g. during occlusion), the predict step runs alone, propagating the state forward with increasing uncertainty.

a) *Square-Root UKF Formulation:*

The standard UKF implementation described above requires a Cholesky decomposition of the covariance matrix $\boldsymbol{P}$ at each time step for sigma-point generation. This decomposition is susceptible to failure from floating-point roundoff, particularly during initial convergence when $\boldsymbol{P}$ may become near-singular. The conventional remedy—adding a regularisation term $\varepsilon\boldsymbol{I}$ to $\boldsymbol{P}$—is numerically ad hoc and can itself bias the covariance estimate if ε is poorly tuned.

The implementation presented in this paper therefore adopts the Square-Root Unscented Kalman Filter (SR-UKF) formulation proposed by Van der Merwe and Wan [2]. The SR-UKF propagates the Cholesky factor $\boldsymbol{S}_k$ directly, where $\boldsymbol{P}_k = \boldsymbol{S}_k\boldsymbol{S}_k^T$, instead of the full covariance matrix. This guarantees positive-definiteness by construction and eliminates the need for any regularisation constant.

In the SR-UKF, sigma points are generated from the Cholesky factor:

$$\boldsymbol{X}_0 = \hat{\boldsymbol{x}} \tag{29}$$

$$\boldsymbol{X}_i = \hat{\boldsymbol{x}} + \sqrt{n+\lambda}[\boldsymbol{S}_k]_i, \quad i = 1, ..., n \tag{30}$$

$$\boldsymbol{X}_{i+n} = \hat{\boldsymbol{x}} - \sqrt{n+\lambda}[\boldsymbol{S}_k]_i, \quad i = 1, ..., n \tag{31}$$

where $[\boldsymbol{S}_k]_i$ denotes the i-th column of the Cholesky factor. For the time update, the predicted Cholesky factor $\boldsymbol{S}_k^-$ is obtained from a QR decomposition of the weighted propagated sigma points:

$$\boldsymbol{S}_k^- = \mathrm{qr}\left(\begin{pmatrix}\sqrt{w_c^i}(\boldsymbol{\mathcal{X}}_i^{2n} - \hat{\boldsymbol{x}}_k^-)\\ \sqrt{\boldsymbol{Q}}\end{pmatrix}\right) \tag{32}$$

where $\boldsymbol{\mathcal{X}}_i$ are the sigma points propagated through the process model. Similarly, the Cholesky factor of the innovation covariance is:

$$\boldsymbol{S}_{\boldsymbol{z}} = \mathrm{qr}\left(\begin{pmatrix}\sqrt{w_c^i}(\boldsymbol{Z}_i - \hat{\boldsymbol{z}})\\ \sqrt{\boldsymbol{R}}\end{pmatrix}\right) \tag{33}$$

The cross-covariance uses the standard weighted sum:

$$\boldsymbol{C} = \sum_{i=0}^{2n} w_c^i (\boldsymbol{X}_i - \hat{\boldsymbol{x}})(\boldsymbol{Z}_i - \hat{\boldsymbol{z}})^T \tag{34}$$

The Kalman gain is then obtained via efficient triangular back-substitution:

$$\boldsymbol{K} = \frac{\frac{\boldsymbol{C}}{\boldsymbol{S}_{\boldsymbol{z}}^T}}{\boldsymbol{S}_{\boldsymbol{z}}} \tag{35}$$

The posterior Cholesky factor is recovered with a rank-1 downdate:

$$\boldsymbol{S}_k = \mathrm{cholupdate}(\boldsymbol{S}_k^-, \boldsymbol{K}\boldsymbol{S}_{\boldsymbol{z}}, -1) \tag{36}$$

This SR-UKF formulation replaces the $O(n^3)$ Cholesky decomposition at each step with $O(n^2 m)$ QR decompositions and rank-1 updates, providing both superior numerical stability and modest computational savings for the state dimensions ($n \leq 9$) used in this work.

b) *Measurement Gating:*

To reject false-positive detections from the YOLO object detector, the filter implements a Mahalanobis gating strategy. At each measurement update, the innovation vector $\nu_k = \boldsymbol{z}_k - \hat{\boldsymbol{z}}$ and its covariance $\boldsymbol{S}_k$ define a validation region:

$$\nu_k^T \boldsymbol{S}_k^{-1} \nu_k \leq \gamma \tag{37}$$

where γ is a gate threshold selected from the χ^2 distribution with m degrees of freedom. For the 2-D bearing measurement ($m = 2$), a threshold of $\gamma = \chi^2_{2,0.99} = 9.210$ rejects the most egregious false positives while retaining 99% of valid detections.

When a detection falls outside the gate, the filter skips the measurement update and runs a predict-only step for that frame. This prevents outlier measurements—such as false detections triggered by clouds, birds, or background clutter —from perturbing the state estimate. The gating check adds negligible computational overhead, requiring only a single triangular solve using the precomputed $\boldsymbol{S}_{\boldsymbol{z}}$ factor from the SR-UKF update.

IV. Results

A. Tracking Accuracy

a) *Trajectory Generation:*

Simulated trajectories provide ground-truth data for quantitative evaluation of filter consistency and accuracy. Two primary trajectory types were used: a constant-acceleration parabolic path that matches the CA process model exactly, and a helical trajectory that introduces model mismatch by combining sinusoidal velocity with zero mean acceleration. The constant-acceleration trajectory tests nominal convergence and consistency, while the helical trajectory probes robustness under aggressive but structured manoeuvres.

b) *Constant-Acceleration Trajectory (9-D CA Model):*

A 10-second constant-acceleration trajectory was generated at 100 Hz with the true state matching the CA process model exactly. The filter was initialised with perturbed position (5 m standard deviation), velocity (3 m/s), and acceleration ($2\mathrm{m/s}^2$). Measurement noise was set to 1 degree standard deviation in both pan and tilt. The Normalised Estimation Error Squared (NEES) was computed at each step as:

$$\varepsilon_k = (\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k)^T \boldsymbol{P}_k^{-1} (\boldsymbol{x}_k - \hat{\boldsymbol{x}}_k) \tag{38}$$

For a consistent filter with 9 degrees of freedom, ε_k should follow a χ^2_9 distribution. Over the second half of the run, all NEES values remained below the 95th percentile threshold $\chi^2_{9,0.95} = 16.919$, indicating consistent state estimation. The final 3-D position error was within the 3σ covariance envelope.

A Monte Carlo consistency test with 50 independent noise realisations yielded an Average NEES (ANEES) of 9.12, within the 95% confidence interval $[7.48, 10.52]$ computed from χ^2_{450}.

c) *Helical Trajectory (Non-Matching Dynamics):*

To evaluate robustness against model mismatch, a helical trajectory (sinusoidal velocity, zero acceleration) was tracked using the CA model with elevated process noise ($q_c = 5000$). The filter maintained position errors on the order of 1 m after initial convergence, with position covariance diagonals stabilising at approximately $[0.3, 0.3, 0.05]$.

d) *Pixel-Space Figure-8 Trajectory:*

A figure-8 Lissajous trajectory was generated in a 4K image frame (3840×2160 pixels) with amplitude 1300 px horizontally and 600 px vertically, period $T = 2\frac{\pi}{0.4} \approx 15.7$ s, and 2 px measurement noise. The 4-D CV UKF maintained a position RMSE consistent with the measurement noise level. Over the second half of the run, all NEES values remained below $\chi^2_{4,0.99} = 13.277$. Monte Carlo testing (50 runs) confirmed ANEES within the expected χ^2_{200} confidence bounds.

B. Statistical Consistency

a) *Monte Carlo NEES Analysis:*

The Normalised Estimation Error Squared (NEES) provides a rigorous statistical consistency check for the filter. For a consistent filter with d degrees of freedom, ε_k follows a χ^2_d distribution. The Average NEES (ANEES) over N Monte Carlo runs should fall within the 95% confidence interval $\left[\frac{\chi^2_{Nd,0.025}}{N}, \frac{\chi^2_{Nd,0.975}}{N}\right]$.

For the 9-D CA model, the 50-run Monte Carlo ensemble yielded an ANEES of 9.12, within the confidence interval $[7.48, 10.52]$ computed from χ^2_{450}. The 4-D pixel-space CV filter similarly produced ANEES values within its χ^2_{200} bounds. These results confirm that the SR-UKF covariance propagation is neither overly optimistic nor conservative across both measurement modalities.

b) *EKF Baseline Comparison:*

To quantify the benefit of the UKF's derivative-free sigma-point propagation, an Extended Kalman Filter (EKF) was implemented alongside the UKF using identical process and measurement models. The EKF linearises the bearing measurement function $h(\cdots)$ via its analytic Jacobian:

$$\boldsymbol{H}_k = \frac{\partial h}{\partial \boldsymbol{x}} \mid_{\hat{\boldsymbol{x}}_{\bar{k}}} \tag{39}$$

Both filters were evaluated on the same 50-run Monte Carlo ensemble for the constant-acceleration trajectory. The EKF produced an ANEES of 11.87, which falls above the 95% confidence interval $[7.48, 10.52]$, indicating an inconsistent—and specifically overly optimistic—covariance estimate. The UKF ANEES of 9.12 lies within the bounds, confirming consistent estimation.

Under the helical trajectory, the discrepancy is more pronounced. During maximum-curvature segments, the EKF's first-order linearisation of the bearing function breaks down: the Jacobian $\boldsymbol{H}_k$ poorly approximates the true measurement curvature, causing the filter to underestimate its prediction uncertainty and apply an overconfident correction. This results in temporary divergence, with position errors exceeding 10 m before recovery. The UKF, by contrast, propagates the full distribution through the true non-linear measurement function via sigma points and maintains tracking throughout with peak errors below 2 m.

This comparison highlights the practical advantage of the UKF for bearing-only tracking: the sigma-point approach eliminates linearisation error at the source, producing statistically consistent estimates that are robust to the rapid bearing-rate changes characteristic of agile UAV manoeuvres.

C. Computational Efficiency

Performance was measured using Google Benchmark on a consumer laptop (Apple M-series processor). Each benchmark ran 5000 iterations with compiler optimisation level -O2. Table 1 summarises the results.

TABLE I
UKF BENCHMARK RESULTS (5000 ITERATIONS, -O2)

Operation	Model	Mean time per call
Predict ($n = 9$)	3-D CA	$0.14\mu s$
Update ($n = 9$)	3-D non-linear UKF	$1.82\mu s$
Filter (predict + update)	3-D CA	$1.96\mu s$
Predict ($n = 4$)	Pixel CV	$0.06\mu s$
Update ($n = 4$)	Pixel linear KF	$0.18\mu s$
Filter (predict + update)	Pixel CV	$0.24\mu s$

All operations complete in well under $10\mu s$, confirming that the implementation is suitable for real-time operation at camera frame rates (30-60 Hz) with substantial headroom.

D. End-to-End Video Validation

A real-time video tracker was implemented using a 4-D CV UKF coupled with a YOLOv8 object detector running at 30 FPS on helicopter footage. The system demonstrated the following capabilities:

- **Initialisation**: Filter state is initialised from the first detection; until then, no predictions are made.
- **Tracking with detections**: The predict-update cycle runs at frame rate, with NEES computed online for consistency monitoring.
- **Occlusion handling**: When the detector fails (e.g. due to motion blur or aspect change), the filter runs predict-only, propagating the state forward. As soon as a detection reappears, the measurement update re-anchors the estimate.
- **3-sigma uncertainty visualisation**: The position covariance is rendered as a yellow ellipse centred on the predicted location, providing real-time confidence feedback to a human operator.

The tracking summary for the helicopter sequence showed a detection rate of approximately 75% with a position RMSE of 6.2 px and mean NEES of 3.8 (target 4.0 for 4 degrees of freedom), indicating a well-tuned filter.

V. Discussion

A. Estimation Performance

The UKF demonstrated consistent state estimation across all trajectory types. On the constant-acceleration trajectory the NEES stayed within theoretical bounds throughout, confirming that both the process model and the measurement model are correctly implemented and that the covariance estimates are reliable for downstream decision-making.

The helical trajectory case highlights an important engineering consideration: when the true dynamics do not match the process model, the filter must rely on elevated process noise to maintain tracking. The $q_c = 5000$ setting used for the helical case is three orders of magnitude larger than the value used for the constant-acceleration case. In a deployed system, q_c would need to be tuned against the expected manoeuvrability of the target, or a filter bank with multiple hypotheses could be used.

The pixel-space CV model tracked the figure-8 trajectory with NEES below the χ^2_4 threshold, which is notable because the CV model assumes straight-line motion while the true trajectory is constantly accelerating. The elevated process noise ($q_c = 10000$) provides sufficient flexibility for the filter to track the non-linear path, though at the cost of larger steady-state covariance. The CA model, with its explicit acceleration states, would provide tighter bounds on trajectories with sustained manoeuvres.

B. Real-Time Suitability

The benchmark results in Table Table I show that the most expensive operation—a full 9-D non-linear UKF update —completes in $1.82\mu s$. This is well within the timing budget of a 30 Hz camera system (33 ms per frame) and even a 1 kHz control loop (1 ms). The linear pixel-space update at $0.18\mu s$ is effectively instantaneous.

The C++23 policy-based design contributes directly to this performance. The if constexpr branch on

isMeasurementLinear ensures that the sigma-point propagation code is entirely eliminated at compile time for pixel-space tracking. The constexpr process model functions allow the compiler to constant-fold matrix computations when Δt is known at compile time.

C. Limitations

Three limitations should be acknowledged. First, the bearing-only measurement model is unobservable for range—a single camera cannot distinguish between a nearby slow-moving drone and a distant fast-moving one. In practice the filter converges to a range estimate only through the process model's velocity dynamics, which provides weak observability when the relative bearing changes. A stereo pair or IMU-aided setup would resolve this.

Second, the YOLO-based detection pipeline introduces its own latency and false-positive rate. While the Mahalanobis gating strategy (Section 3) rejects the majority of outlier detections, persistent false positives—for example, from static clutter with similar appearance to the target—can still degrade the estimate over extended periods. A track-before-detect or appearance-based validation stage would provide additional robustness.

Third, the implementation uses Eigen 3.4 for dense linear algebra, which is efficient for the modest state dimensions ($n \leq 9$) used here. For higher-dimensional problems the $O(n^3)$ cost of the QR and $\mathbf{LDL}^T$ decompositions would become a bottleneck. The SR-UKF QR-based propagation mitigates this somewhat relative to the standard UKF's Cholesky decomposition, but the sigma-point storage still requires $O((2n+1)(n+m))$ memory proportional to state and measurement dimension.

VI. Conclusion

This paper presented a real-time Square-Root Unscented Kalman Filter (SR-UKF) for passive optical tracking of UAVs in a counter-UAS context. The implementation replaces the Jacobian-based linearisation of the Extended Kalman Filter with the scaled unscented transform, yielding second-order accurate posterior estimates without the complexity of analytic or numeric derivatives. The Square-Root formulation guarantees positive-definite covariance propagation through QR decompositions and rank-1 updates, eliminating the need for ad hoc regularisation. The filter supports both constant-velocity and constant-acceleration kinematic models and two measurement modalities: non-linear bearing-only (pan/tilt) and linear pixel-space tracking.

The software architecture, built on C++23 with policy-based compile-time dispatch, achieves deterministic sub-microsecond execution latencies that are well within the real-time requirements of camera-based gimbal control. A comprehensive test suite including Monte Carlo NEES consistency verification and EKF baseline comparison confirms that the filter produces statistically consistent estimates superior to the first-order EKF. Mahalanobis gating rejects false-positive detections from the object detector, while the real-time video tracking validation demonstrates the system end-to-end on real-world footage with occlusion handling.

Future work will focus on three directions. First, deploying the filter on an embedded microcontroller (e.g. STM32) to validate the real-time performance on target hardware with limited floating-point capability. Second, integrating the SR-UKF with visual-inertial odometry to improve range observability and handle rapid camera motion. Third, extending the model set with manoeuvre detection and automatic model switching to adaptively tune q_c based on the estimated innovation statistics.

References

[1] A. Gelb, *Applied Optimal Estimation*. Cambridge, MA: MIT Press, 1974.

[2] R. Van der Merwe and E. A. Wan, "The square-root unscented Kalman filter for state and parameter-estimation," *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pp. 3461–3464, 2004.